

Force-Control Stack — Controller Summary

Franka Panda · MuJoCo sim + real hardware

1. Overview

The stack runs a 1 kHz control loop on the Franka Panda 7-DoF arm. A state machine sequences the robot through distinct phases; each phase selects either **joint-PD** or **Cartesian impedance** as the active controller. An admittance outer loop on the world-z force regulates contact during the force-hold and slide phases.

2. State Machine (core/state_machine.py · JointMoveStateMachine)

User interaction (Enter key presses) gates transitions between phases. Each phase's *update()* call returns a **ControlCommand** with `mode ∈ {joint, cartesian, failed}` that the backend dispatches to the appropriate controller.

Phase	Controller	Entry condition	Exit condition
HOLD	Joint PD (q_hold)	Boot / IK not yet solved	User presses Enter
WAIT_USER_CONFIRM	Joint PD (q_hold)	IK solved	User presses Enter
MOVE_JOINT	Joint PD (min-jerk)	Enter pressed	t_elapsed ≥ move_time
TARE	Joint PD (q_goal)	Move complete	tare_duration elapsed
WAIT_APPROACH_CONFIRM	Joint PD (q_goal)	Tare complete	User presses Enter
APPROACH	Cartesian (descent)	Enter pressed	Fz ≥ threshold (5 consecutive)
FORCE_HOLD	Cartesian (admittance)	Contact detected	User presses Enter (slide enabled)
WAIT_SLIDE_CONFIRM	Cartesian (admittance)	FORCE_HOLD entered	User presses Enter
SLIDE	Cartesian (admittance + xy trajectory)	Enter pressed	distance covered
FAILED	—	IK fail / missing pose	Loop terminates

3. Joint PD Controller (core/controller.py · JointPDController)

Used during HOLD, MOVE_JOINT, TARE, and WAIT phases. Stateless — no integrator.

$$\tau = K_p \cdot (q_{des} - q) + K_d \cdot (dq_{des} - dq)$$

Parameter	Sim value	Units	Notes
Kp (joints 1–4)	200, 200, 200, 200	Nm/rad	Position gain
Kp (joints 5–7)	80, 80, 40	Nm/rad	Distal joints, softer
Kd (joints 1–4)	20, 20, 20, 20	Nm·s/rad	Velocity gain

Kd (joints 5–7)	8, 8, 4	Nm·s/rad	
-----------------	---------	----------	--

Min-jerk trajectory (MOVE_JOINT)

Position setpoints follow a 5th-order polynomial that produces zero velocity and acceleration at both endpoints:

$$s(t) = 10\tau^3 - 15\tau^4 + 6\tau^5, \quad \tau = t / T$$

$$\dot{s}(t) = (30\tau^2 - 60\tau^3 + 30\tau^4) / T$$

4. Cartesian Impedance Controller (core/controller.py · CartesianImpedanceController)

Active during APPROACH, FORCE_HOLD, WAIT_SLIDE_CONFIRM, and SLIDE. No gravity/coriolis term is added here — the backend handles those separately.

$$F = Kp_pos \cdot (x_des - x) + Kd_pos \cdot (dx_des - v)$$

$$T = Kp_ori \cdot e_ori + Kd_ori \cdot (w_des - w)$$

$$\tau = J^T \cdot [F, T]$$

DOF	Kp (K)	Kd (D)	Units
x, y, z (translation)	500, 500, 500	50, 50, 50	N/m ; N·s/m
rx, ry, rz (rotation)	30, 30, 30	5, 5, 5	Nm/rad ; Nm·s/rad

Orientation error (world frame):

$$Re = R_des \cdot R^T, \quad e_ori = 0.5 \cdot \text{vee}(Re - Re^T)$$

Gravity & payload compensation (added by backend, not controller):

Sim: $\tau_grav = G_zero(q)$ (arm-only model) + $\Delta G_payload(q)$ (full – zero model delta).

Hardware: libfranka handles arm gravity internally; backend adds $\Delta G_payload(q)$ only.

$$\tau_total = \tau_grav + \tau_task + \text{null-space term}$$

Torque rate is clamped to max_torque_rate = 1.0 Nm per 1 ms tick.

5. Contact Force Regulation (core/state_machine.py · _update_force_z)

An admittance outer loop converts Fz error to a z-reference velocity, which is fed as $dx_des[2]$ and $x_des[2] = z_ref$ to the Cartesian impedance controller. Separate gain sets are used for FORCE_HOLD vs SLIDE.

$$e = Fz - F_desired$$

$$\dot{z} = Kp_f \cdot e + Kd_f \cdot \Delta e / dt \quad (\text{clamped to } \pm \text{max_z_speed})$$

$$z_ref += \dot{z} \cdot dt \quad (\text{clamped to } [z_contact - \text{max_depth}, z_contact + \text{retract}])$$

Parameter	FORCE_HOLD (sim)	SLIDE (sim)	Units
F_desired	3.0	3.0	N
Kp_f	0.0005	0.05	m/s per N
Kd_f	0.0	0.005	m per N
max_z_speed	0.0015	0.05	m/s
max_depth	0.05	0.05	m (below z_contact)
retract_allowance	0.01	0.01	m (above z_contact)

6. Approach Phase (_update_approach)

Open-loop descent along world -Z at constant speed until contact is confirmed.

Parameter	Sim value	Units
approach_speed	0.05	m/s

contact_fz_threshold	2.0	N (Fz must exceed this)
contact_confirm_samples	5	consecutive ticks above threshold

7. Slide Phase (`_update_slide`)

The slide runs simultaneous z force-hold (SLIDE gains) and a constant-speed xy trajectory with a min-jerk velocity ramp. A tangential velocity-error integral corrects for friction-induced lag.

```
s_ref(t) = clamp(∫ v_ref dt, 0, distance)
v_ref(t) = speed · minJerk(t - t0, ramp_time) [ramps from 0 to speed]
v_cmd = v_ref + ki_vel · ∫(v_ref - v_actual) dt (clamped)
x_des = p_start + u_slide · s_ref
```

Parameter	Sim value	Units
direction_xy	[0, 1]	world-frame XY (auto-normalised)
speed	0.01	m/s (after ramp)
ramp_time	1.0	s (min-jerk ramp)
distance	0.1	m (then hold in place)
ki_vel	20.0	integral gain on v-error
vel_int_limit	0.003	m/s (anti-windup)

8. FT Sensor Pipeline (`core/ft_processor.py · FTProcessor`)

Three-stage pipeline applied each tick:

```
1. Sign flip : w_signed = sign · w_raw (sign = -1 in sim)
2. Bias remove : w_unbiased = w_signed - bias (bias set by tare())
3. Low-pass : w_out = α · w_unbiased + (1-α) · w_prev (α = 1.0 in sim = no filter)
```

Tare procedure: after MOVE_JOINT completes the robot holds the goal pose for *tare_duration* seconds (2 s in sim).

All valid FT samples are averaged and saved as the bias, zeroing out the tool's gravity wrench at that orientation. All subsequent readings are pure contact force.

9. Forward Kinematics (`core/kinematics.py · SiteKinematics`)

Uses a private scratch MjData driven with measured joint positions. Reads the world-frame pose and spatial Jacobian of the controlled site (*stick_tip*) via the minimal MuJoCo pipeline (no dynamics, no collision).

```
mj_kinematics(model, data) → site_xpos, site_xmat
mj_comPos(model, data) → cdof terms needed by mj_jacSite
mj_jacSite(...) → Jp (3×7), Jr (3×7)
v = Jp · dq, w = Jr · dq, J = stack([Jp, Jr])
```

10. IK Solver (`core/ik.py · solve_ik_dls`)

Damped Least Squares (DLS / Levenberg–Marquardt) IK run once at startup on a scratch MjData — the live simulation state is never corrupted.

```
e = [x_des - x_tip ; e_ori] (6-vector pose error)
Δq = J+ (J J+ + λ2 I)-1 e (DLS step)
q ← q + Δq, iterate until ||e|| < tol
```

Target orientation: R_{ref} (site at home keyframe) rotated by $target_euler_xyz = [30^\circ, 0^\circ, 0^\circ]$ expressed as intrinsic XYZ Euler angles.

11. Trajectory Logging (core/data_logger.py · TrajectoryLogger)

RT-safe: a fixed NumPy buffer (*capacity* = 1 500 000 rows $\approx$ 25 min at 1 kHz) is preallocated at startup. Each tick writes one row by index — no allocation or disk I/O in the hot path. The CSV is flushed once after the loop exits (or on Ctrl+C via try/finally).

Column	Description	Unit
t	Simulation / wall time since loop start	s
x, y, z	Tip world-frame position (stick_tip site)	m
vx, vy, vz	Tip world-frame linear velocity ($J_p \cdot dq$)	m/s
Fz	Processed contact force, world z (wrench[2])	N
phase	State machine phase name (e.g. FORCE_HOLD)	string

12. Key File Map

File	Role
core/state_machine.py	JointMoveStateMachine — all phase transitions & force loop
core/controller.py	JointPDController, CartesianImpedanceController
core/ft_processor.py	FTProcessor — sign / bias / low-pass pipeline
core/kinematics.py	SiteKinematics — FK + Jacobian via scratch MjData
core/data_logger.py	TrajectoryLogger — RT-safe per-tick CSV writer
core/ik.py	solve_ik_dls — DLS IK solver, min_jerk, euler helpers
backends/mujoco_backend.py	MuJoCo sim: viewer loop, FT read, log flush on exit
backends/franka_backend.py	Real robot: 1 kHz RT loop, ROS/SHM FT, safety limits
sensors/ft_mujoco.py	FTMuJoCo — reads named MuJoCo force/torque sensors
configs/sim.yaml	All tunable parameters for the simulation run
visualization/plot_log.py	Post-run analysis: 5 matplotlib figures from CSV log
visualization/live_monitor.py	Watches data/logs/ and auto-plots when new CSV appears